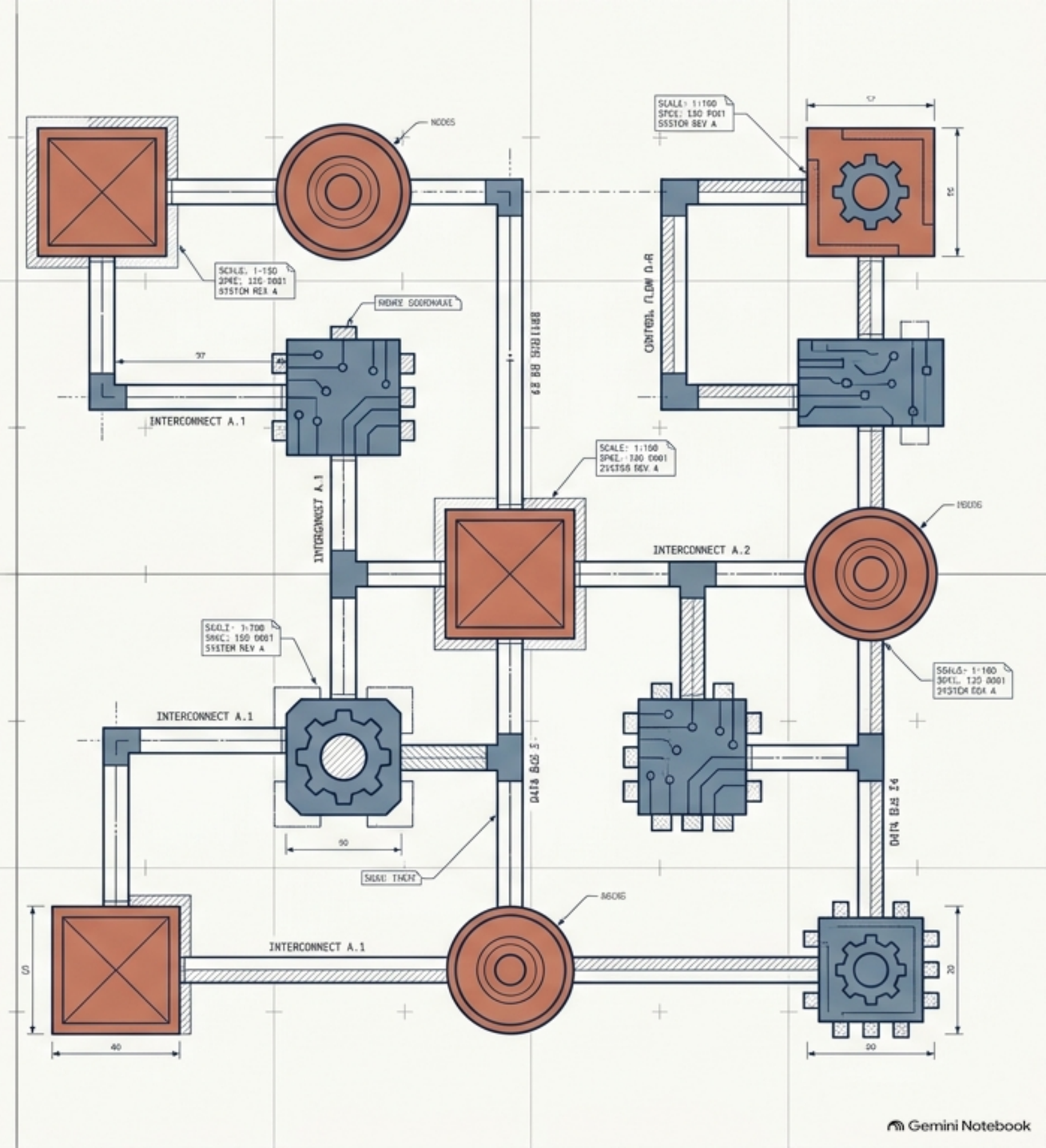
Agentic Architecture

Blueprinting Multi-Agent Ecosystems for the Enterprise

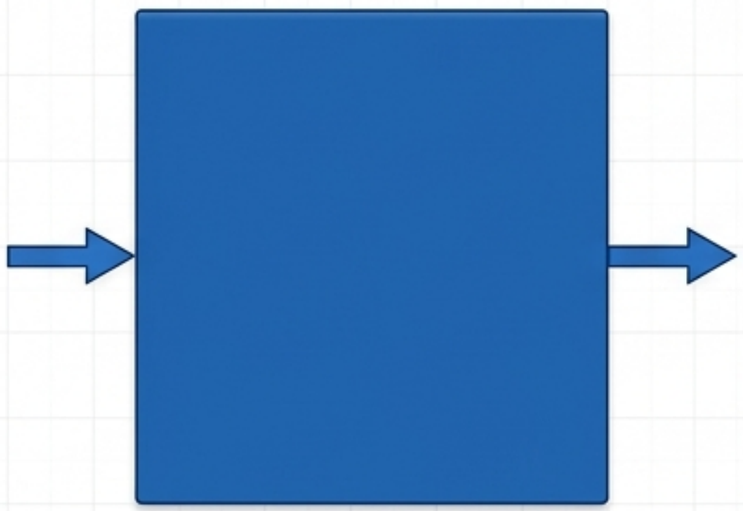
VERSION: 1.0 | TARGET: ARCHITECTURE & SYSTEMS DESIGN



The Architectural Shift

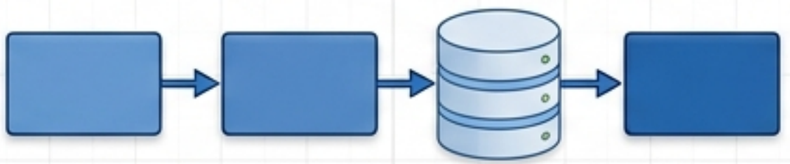
Single-shot LLM calls are insufficient for complex, non-deterministic enterprise tasks. Agentic architecture separates concerns, assigning specialized responsibilities to distributed nodes.

Monolithic



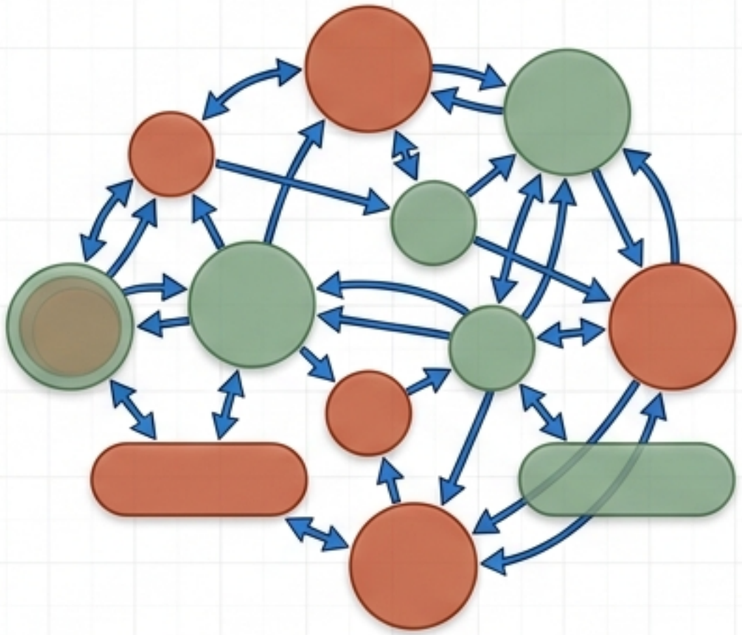
Stateless / Predictable / Rigid

Compound System



Programmatic / Hardcoded Control Logic

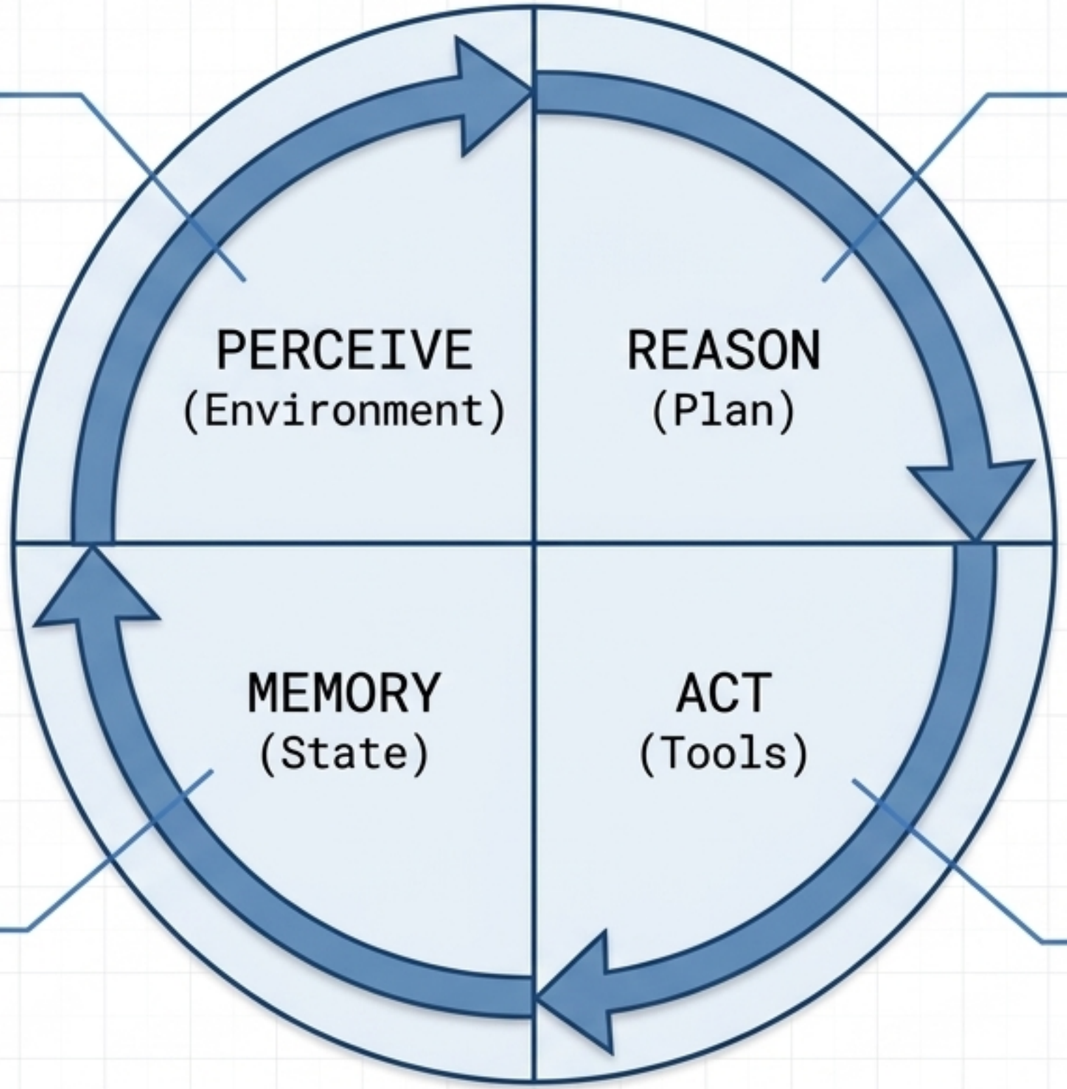
Agentic System



Autonomous / Adaptive / Stateful

Anatomy of an Autonomous Agent

The LLM acts as the core reasoning unit, analyzing inputs and breaking high-level goals into multi-step plans.



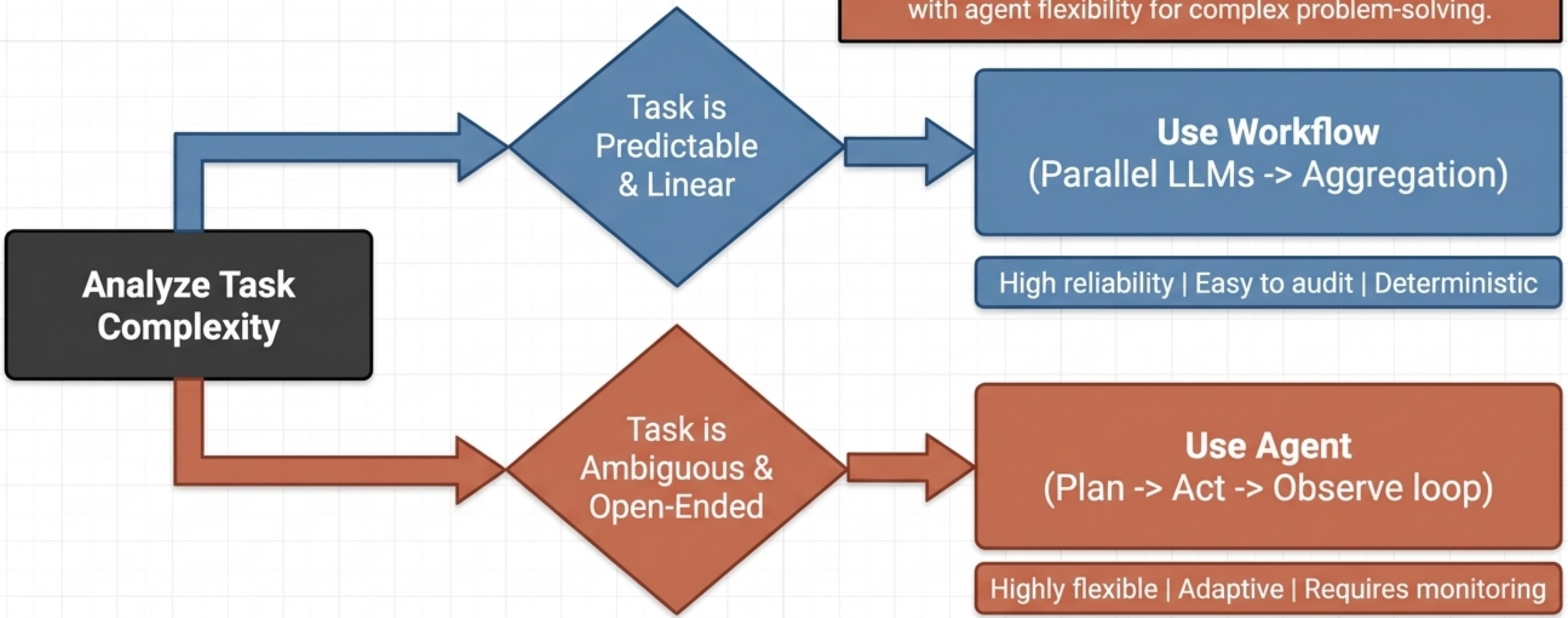
Consistes the LLM ratapning miutate metntorising inputs and posbians multi-seouving prontenenonment.

Persistent context storage prevents catastrophic forgetting and enables iterative problem-solving across the loop.

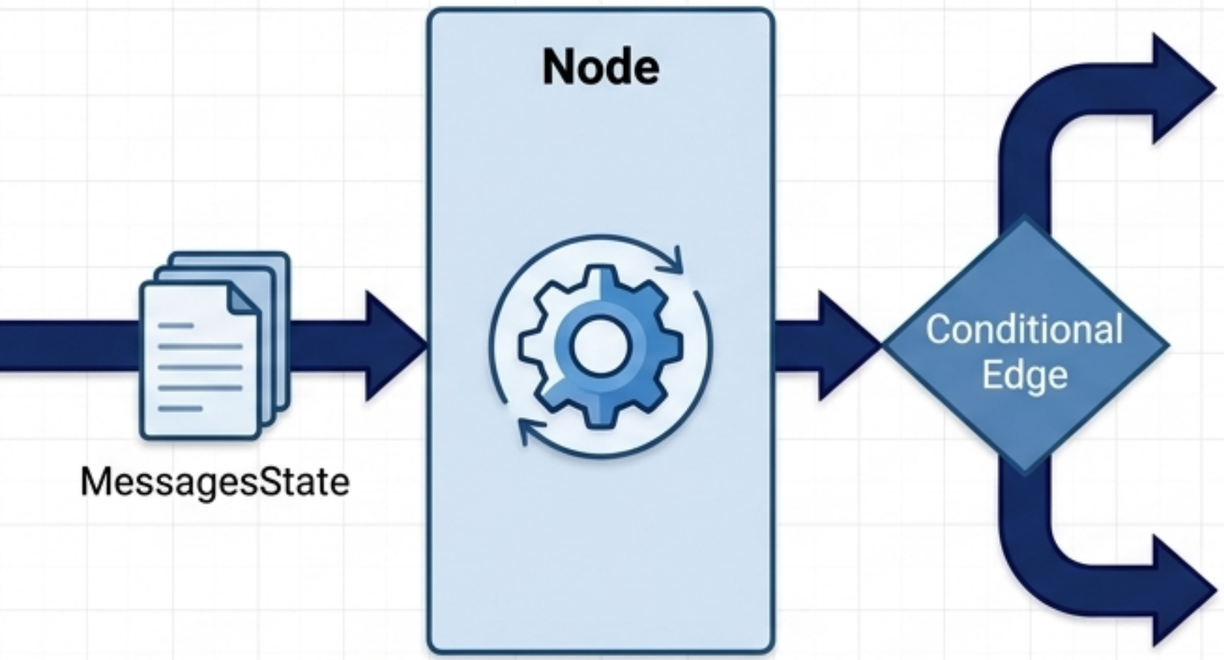
Embedded tool calling (Python, SQL, APIs) allows the agent to mutate its environment.

Architectural Heuristic: Agent vs. Workflow

Not every problem requires an AI agent. Hybrid architectures combine workflow reliability for routine steps with agent flexibility for complex problem-solving.



LangGraph: The Stateful Blueprint




```
class State(TypedDict):
    string_field: str
    string_list_field: List[str]
    int_field: int
    input_field: str
    output_field: Output
```

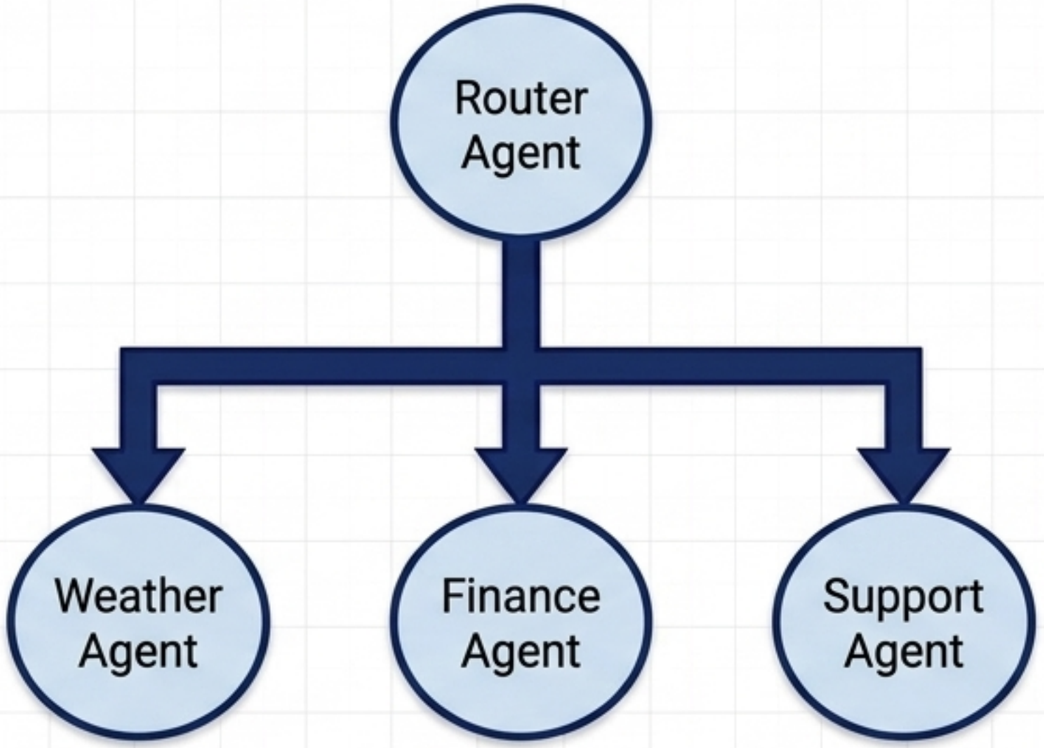
State: The shared persistent memory. Acts as a strict data contract between nodes.

Nodes: Plain Python functions that execute work and return state updates.

Edges: Directed connections defining execution order (including conditional routing).

Core Design Patterns: Flow Control

Sequential Profile
Chain agents in a fixed order. Best for multi-step data pipelines requiring ordered execution.



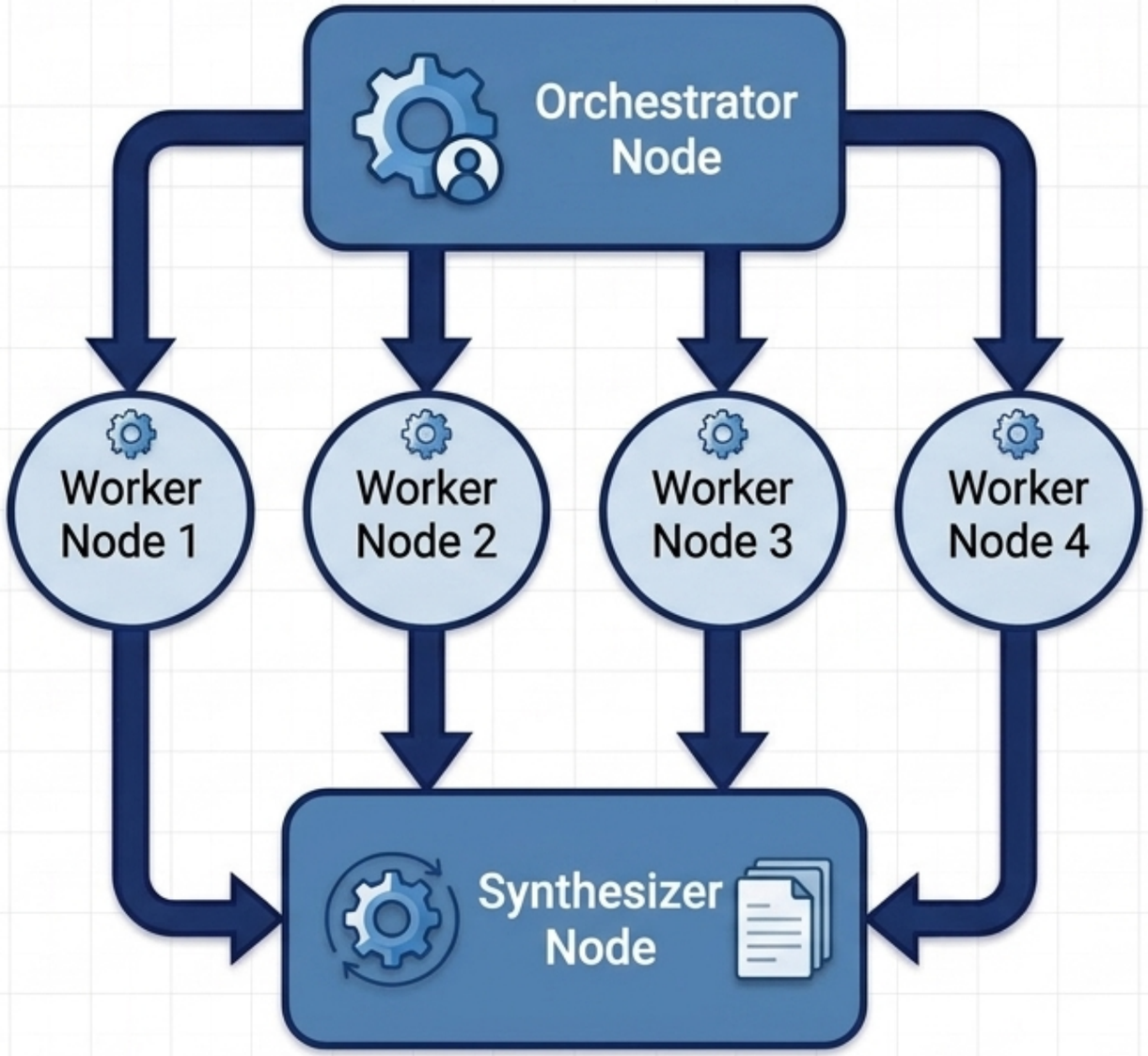
Routing Profile
Dispatch inputs dynamically based on user intent. Centralizes decision logic, ensuring only the necessary specialized agents consume compute.

Core Design Patterns: Orchestrator-Worker

Mechanics

The Orchestrator delegates responsibilities to specialized worker agents based on task complexity.

Workers execute concurrently.

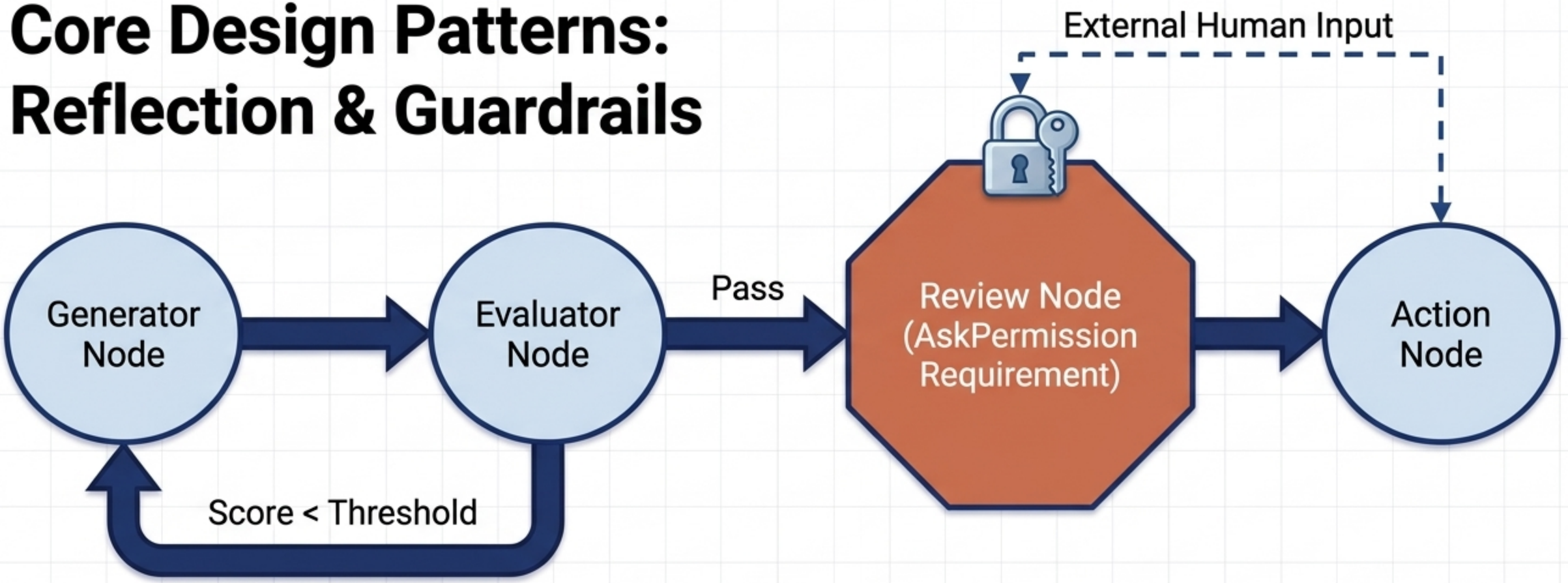


System Impact

Enables dynamic, scalable parallel processing.

Crucial for massive scale (e.g., analyzing 600,000 documents via multi-agent pipelines).

Core Design Patterns: Reflection & Guardrails

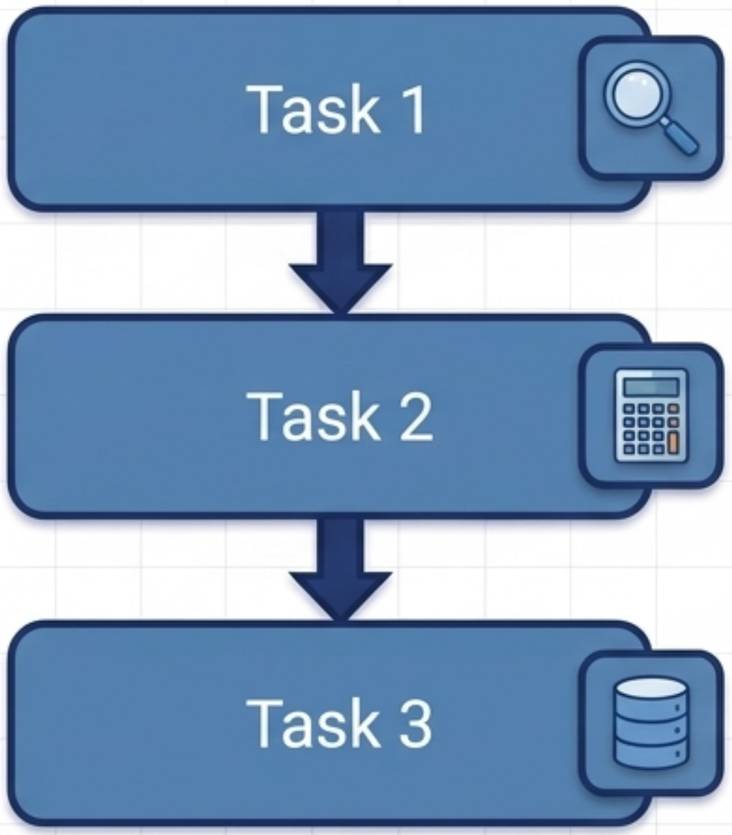


Self-Correction
Evaluator nodes critique outputs, forcing a revision cycle until a quality threshold is met.

Human-in-the-Loop
Execution pauses at strict checkpoints, awaiting human approval before high-stakes actions are taken. Essential for SecOps.

Tool Execution Architecture

Agent-Centric Execution: High autonomy.
The agent dynamically decides which tool to use based on reasoning. Highly flexible, but harder to trace and debug.

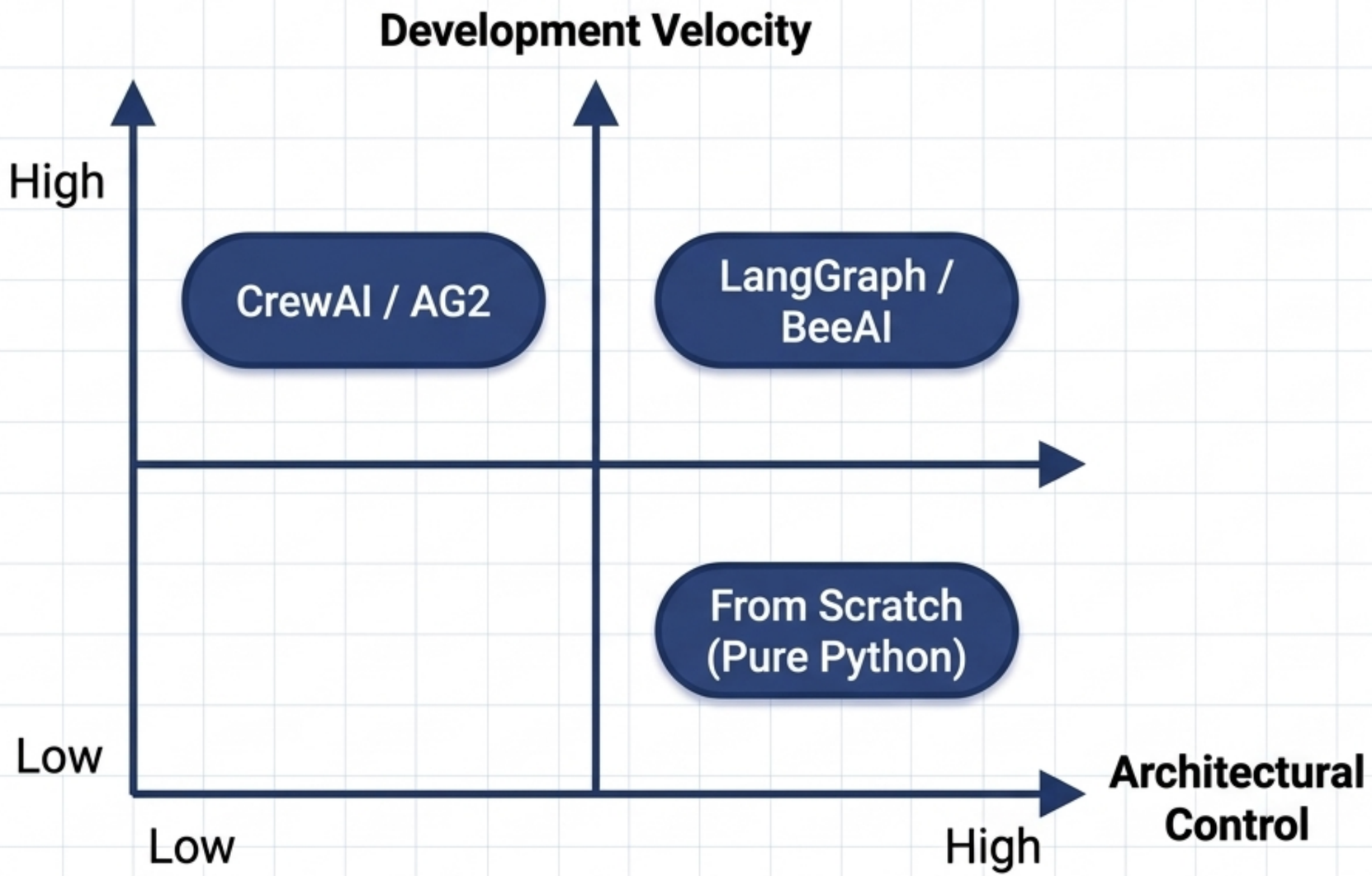


Task-Centric Execution: High control.
Tools are bound to specific tasks. The agent follows a predefined sequence. Highly predictable, ensuring strict task-tool isolation and perfect traceability.

The Multi-Agent Framework Matrix

Framework	Paradigm	Best For	Control Level	Production Readiness
LangGraph	State/Graph paradigm	Ultimate control & custom state	 Max	 Max
CrewAI	Role-playing paradigm	Rapid prototyping & team simulations	 Medium	 Medium
AG2 (AutoGen)	Conversational paradigm	Chat-driven & oversight workflows	 Medium	 High
BeeAI	Enterprise/ReAct paradigm	Scalable, secure deployments	 High	 Max

Build vs. Frameworks



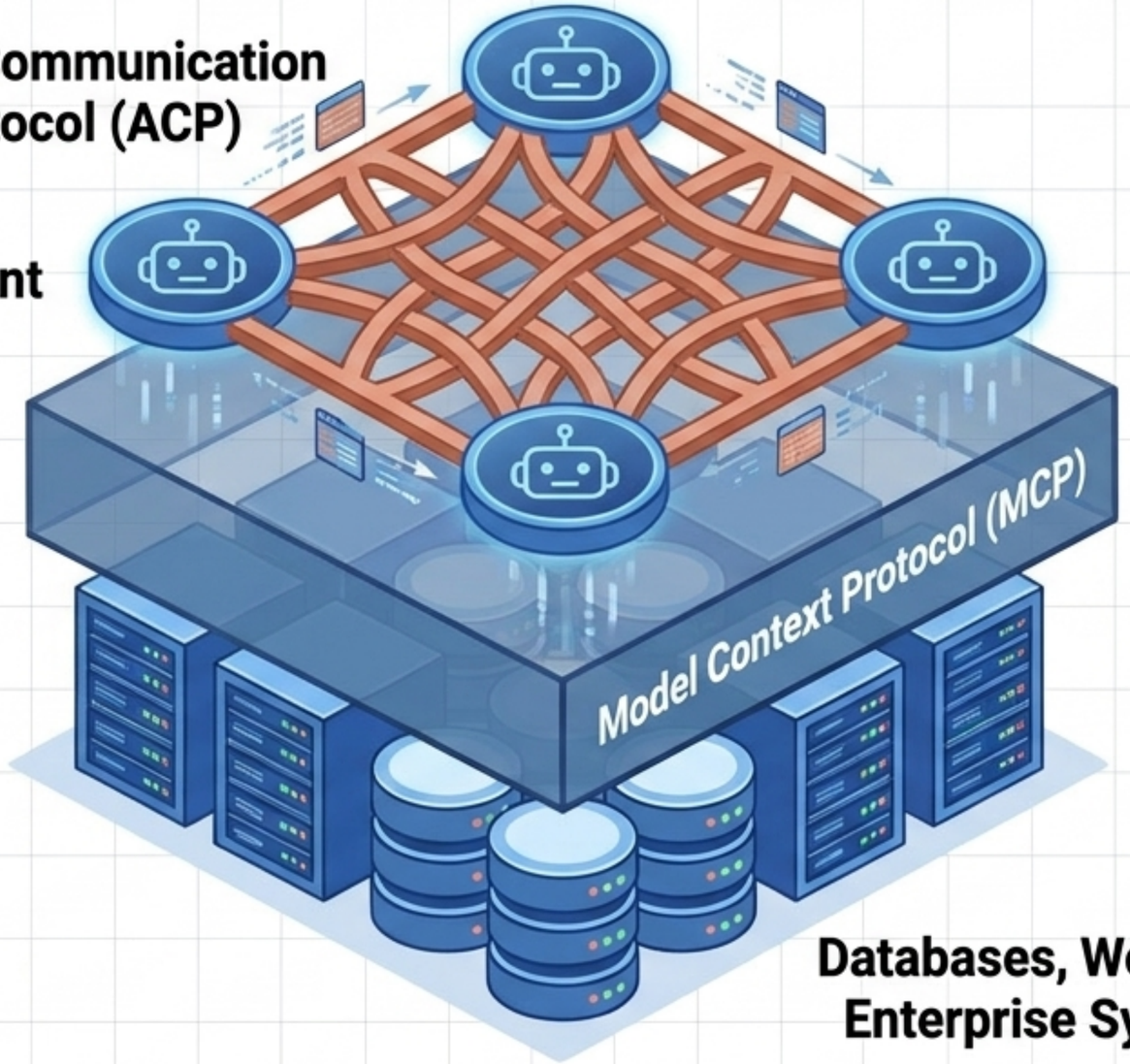
From Scratch

Deepest understanding, manual API calls, explicit error handling. High overhead, absolute control.

Frameworks

Built-in state management, tracing, and pre-built memory abstractions. Vastly reduced boilerplate at the cost of opinionated architectural constraints.

Ecosystem Protocols: MCP & ACP



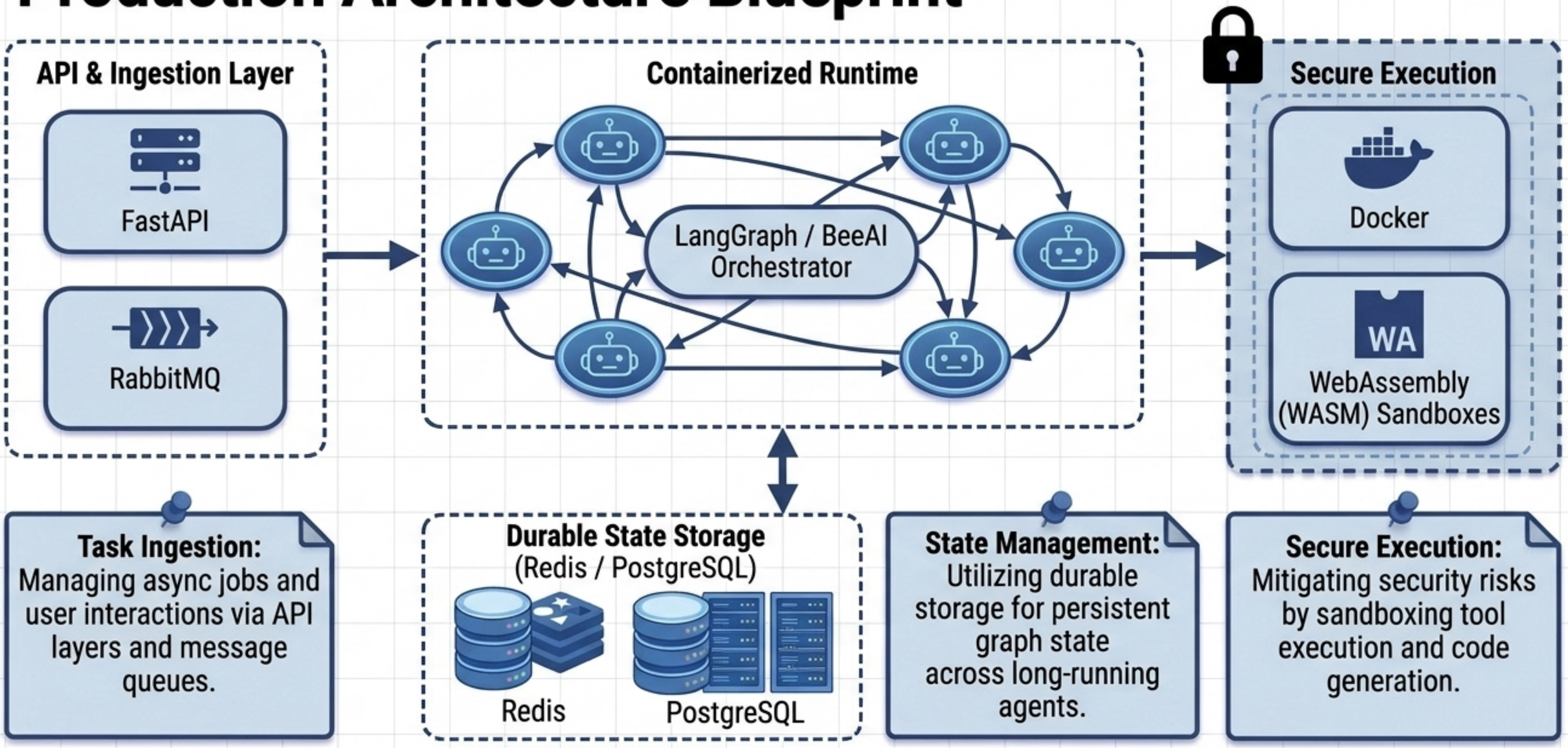
Model Context Protocol (MCP)

The universal open standard connecting agents to external data and tools without custom integrations.

Agent Comm. Protocol (ACP)

The universal standard for agent-to-agent communication, allowing disparate systems to discover, collaborate, and execute cross-framework workflows.

Production Architecture Blueprint



Architectural Imperatives

1

Start Stateless.

Avoid Premature Autonomy. Add task complexity and agentic loops only after confirming reliable programmatic performance.

2

Lock the Tools.

Enforce Strict Contracts. Use task-centric tool execution and strict schemas
schemas (Pydantic) to ensure predictable, traceable interactions.

3

Graph the State.

State is King. Treat multi-agent systems as complex state machines. Centralize memory and state management to guarante-than visibility and control.